SOFTWARE CONSTRUCTION AND DEVELOPMENT

SOFTWARE CONSTRUCTION (FROM PREVIOUS LECTURE)

- **As the figure indicates, *construction is mostly coding and debugging* but also involves detailed design, construction planning, unit testing, integration, integration testing, and other activities.**

SOFTWARE CONSTRUCTION (FROM PREVIOUS LECTURE)

Here are some of the specific tasks involved in construction:

Verifying that the groundwork has been laid so that construction can proceed successfully

- Determining how your code will be tested
- Designing and writing classes and routines
- Creating and naming variables and named constants
- Selecting control structures and organizing blocks of statements
- Unit testing, integration testing, and debugging your own code
- Reviewing other team members' low-level designs and code and having them review yours
- Polishing code by carefully formatting and commenting it
- Integrating software components that were created separately
- Tuning code to make it faster and use fewer resources

WHY SOFTWARE CONSTRUCTION IS IMPORTANT (FROM PREVIOUS LECTURE)

- Construction is a large part of software development
- Construction is the central activity in software development
- With a focus on construction, the individual programmer's productivity can improve enormously
- Construction's product, the source code, is often the only accurate description of the software
- Construction is the only activity that's guaranteed to be done

INTRODUCTION

- The term software construction refers to the detailed creation of working software through a combination of coding, verification, unit testing, integration testing, and debugging.
- Quality (or lack thereof) is very evident in the construction products
- Construction highly related to Computer Science due to
  - Use of algorithms
  - Detailed coding practices

SOFTWARE CONSTRUCTION FUNDAMENTALS

- Minimizing complexity
- Anticipating change
- Constructing for verification
- Reuse
- Standards in software construction

MINIMIZING COMPLEXITY

- Humans are severely limited in our ability to hold complex information in our working memories
- As a result, minimizing complexity is one the of strongest drivers in software construction
- Need to reduce complexity throughout the lifecycle
- As functionality increases, so does complexity

MINIMIZING COMPLEXITY

- Accomplished through use of standards
- Examples:
  - UML for modeling all aspects of complex systems
  - High-order programming languages such as C++ and Java
  - Source code formatting rules to aid readability

ANTICIPATING CHANGE

- Software changes over time
- Anticipation of change affect how software is constructed
- This can effect
  - Handling of errors
  - Source code organization
  - Code documentation
  - Coding standards

CONSTRUCTING FOR VERIFICATION

- Construct software that allows bugs to be easily found and fixed
- Examples:
  - Enforce coding standards
    - Helps support code reviews
  - Unit testing
  - Organizing code to support automated testing
  - Restricted use of complex or hard-to-understand language structures

REUSE

- Reuse refers to using existing assets in solving different problems.
- In software construction, typical assets that are reused include libraries, modules, components, source code..
- Reuse is best practiced systematically, according to a well-defined, repeatable process.
- Systematic reuse can enable significant software productivity, quality, and cost improvements.

STANDARDS IN CONSTRUCTION

Standards which directly affect construction issues include:

- Programming languages
  - E.g. standards for languages like Java and C++
- Communication methods
  - E.g. standards for document formats and contents
- Platforms
- Tools
  - E.g. diagrammatic standards for notations like the Unified Modeling Language